Generative Adversarial Networks

Structured Notes

Autoencoders, Variational

Autoencoders, Discriminative vs Generative Models,
GAN Architecture, Loss Functions, and GAN Variants

Contents

1 Introduction to Generative Adversarial Networks **2**
1.1 Motivation: Why GANs Are Needed . . . 2
1.2 The Synthetic Image Generation Problem . . . 2

2 Review of Autoencoders **2**
2.1 Autoencoder Architecture . . . 2
2.2 Applications of Autoencoders . . . 2

3 Variational Autoencoders (VAE) **3**
3.1 VAE Architecture . . . 3
3.2 Role of Epsilon (ϵ) . . . 3
3.3 Key Properties of VAE . . . 3
3.4 Disadvantages of VAE . . . 4

4 Discriminative vs Generative Models **4**
4.1 Limitations of Discriminative Models . . . 4
4.2 Generative Models . . . 4
4.2.1 Example . . . 4
4.3 Why Generative Models Are Needed . . . 4

5 GAN Architecture **4**
5.1 Components . . . 5
5.1.1 Training Samples (Real Images) . . . 5
5.1.2 Generator . . . 5
5.1.3 Discriminator . . . 5
5.2 The Adversarial Training Process . . . 5

6 GAN Loss Function **5**
6.1 Binary Cross-Entropy Foundation . . . 5
6.2 Discriminator Loss . . . 6
6.3 Generator Loss . . . 6
6.4 Combined Min-Max Objective . . . 6

7 Advantages of GANs **6**

8 Challenges of GANs **6**
8.1 Vanishing Gradient Problem . . . 7
8.2 Mode Collapse . . . 7
8.3 Solutions to GAN Challenges . . . 7

9 Types of GANs **7**
9.1 Vanilla GAN . . . 7
9.2 DCGAN (Deep Convolutional GAN) . . . 7
9.3 Wasserstein GAN (WGAN) . . . 8

10 Summary **8**

1 Introduction to Generative Adversarial Networks

Generative Adversarial Network (GAN)

A **Generative Adversarial Network (GAN)** is a type of neural network architecture that generates output by using the concept of **adversarial learning**. The network considers an adversarial learning framework to produce outputs, and this adversarial competition between two sub-networks is what gives GANs their name.

1.1 Motivation: Why GANs Are Needed

The primary motivation for GANs stems from the need to generate images (or other data) artificially for various applications:

- In real-life scenarios, collecting large quantities of images is **very expensive**.
- Training deep learning architectures requires both input images and corresponding output images (e.g., noisy image → clean image, input image → segmentation output).
- Labeling images requires human effort and is costly — different variations (road images, cycle images, mountain images, book images) each need corresponding labels.
- Robust learning of deep learning architectures requires diverse training data with many variations.

1.2 The Synthetic Image Generation Problem

Since real-life image capture is expensive, the alternative is **synthetic image generation**. There are two approaches:

1. **Manual synthetic generation**: Arranging different objects in scenes manually. This is costly because:
   - High payment required for human labor
   - Additional labeling effort needed
   - Cannot create diverse real-world data variations
2. **AI-based synthetic generation**: Using AI algorithms to generate synthetic images automatically. The generated images should provide a richer representation of objects and better generalization.

2 Review of Autoencoders

Autoencoder

An **Autoencoder** is a type of neural network (or CNN) architecture that learns to **compress** and **reconstruct** input data. It removes irrelevant details from the input and reconstructs a better output corresponding to a target task.

2.1 Autoencoder Architecture

The autoencoder consists of two main components:

1. **Encoder**: Takes the input image X and represents it in a **compressed form** in the **latent space** (lower-dimensional representation).
2. **Decoder**: Takes the latent space representation and reconstructs the target output image.

2.2 Applications of Autoencoders

- **Denoising**: Input is a noisy image; the latent space removes noise information, and the decoder produces a clean image.
- **Segmentation**: Input image is encoded; the latent space identifies homogeneous regions (roads, trees, buildings), and the decoder outputs a segmented image with single-class labels.
- **Medical imaging**: MRI images are encoded; the latent space isolates important features (e.g., tumor regions), and the decoder produces output highlighting only the regions of interest.

Autoencoder Limitation

One of the biggest challenges with autoencoders is that they always produce a **blurry output**. This happens because information is lost during compression in the latent space.

3 Variational Autoencoders (VAE)

Variational Autoencoder (VAE)

A **Variational Autoencoder (VAE)** is a variant of the autoencoder that is a **probabilistic generative model**. It uses probability concepts (specifically normal distribution) to generate different output images, rather than producing a single deterministic output.

3.1 VAE Architecture

The VAE architecture works as follows:

1. **Input**: An input X (image, signal, or other data) is fed into the system.
2. **Encoder**: The encoder produces **two** output vectors:
   - **Mean vector** (μ): Represents a blurred/averaged version of the input — captures the general structure.
   - **Standard deviation vector** (σ): Represents diversification of the input — captures the range of variation.
3. **Latent variable**: The two vectors are combined as:

$$z = \mu + \sigma \cdot \epsilon$$

   where $\epsilon \sim \mathcal{N}(0, 1)$ is a random variable drawn from a standard normal distribution.
4. **Decoder**: The latent variable z is passed through a decoder to generate the output image.

Key Clarification

The mean (μ) and standard deviation (σ) vectors are **not** calculated from the input X. They are two separate encoded outputs produced by the encoder neural network. If the input is an image, both μ and σ are also image-sized vectors. The naming convention reflects their **roles**: μ captures the average/blurred representation, while σ captures the diversity/spread.

3.2 Role of Epsilon (ϵ)

The reparameterization trick uses:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

By sampling different values of ϵ from the standard normal distribution, multiple latent vectors z can be generated. Each z corresponds to a **diversified version** of the input image.

For example, given an image of a cat, the VAE can generate different types of cats: black cat, white cat, patched cat, brown cat — all by varying the ϵ value.

3.3 Key Properties of VAE

- Rather than a single latent space (as in standard autoencoders), the VAE has **two latent representations** combined with a normal distribution.
- The combined z is the **bottleneck** (latent space) of the VAE.
- The VAE generates different output variations by varying ϵ, hence the name "variational."

3.4 Disadvantages of VAE

1. **Reconstruction loss**: Since several variations are generated with different ϵ values, reconstruction loss is present.
2. **Latent loss**: Loss occurs in the latent space representation.
3. **Blurry output**: While better than standard autoencoders, VAEs still produce somewhat blurred outputs due to the mean component.

4 Discriminative vs Generative Models

4.1 Limitations of Discriminative Models

Autoencoder and VAE architectures, despite being called generative, are fundamentally **discriminative** in nature:

- They take an input image and generate output **based on the given input data**.
- They cannot create entirely new types of images on demand.
- They cannot generate complex scenes from text descriptions (e.g., "a horse moving near a beautiful lake" or "a greeting card with five friends").
- They **segregate** or **discriminate** parts of the input, removing unrequired parts — hence the name discriminative.

4.2 Generative Models

Generative Model

A **generative model** takes a set of training examples as input, learns the **underlying probability distribution** of the training data, and generates **new samples** from that distribution. Rather than only predicting class labels (discrimination), it creates output samples that have similarity or distribution mapping with the training examples.

4.2.1 Example

Given a training image of a horse with a lake background in winter, a generative model can produce a new image of a zebra with a similar environment in summer — by approximating the probability distribution of the training samples and generating new samples from it.

4.3 Why Generative Models Are Needed

1. **Complex probability distributions**: Can model distributions that were not previously possible (e.g., multiple animal species in a single picture).
2. **Missing data reconstruction**: Can predict and fill in missing data (e.g., reconstructing lost audio segments, removing occluding objects from images).
3. **Multi-modal output**: Can generate combinations of images, voice, video, and other modalities.
4. **Realistic image generation**: Can create synthetic images that look so realistic they are indistinguishable from real images.

Connection to Modern AI

Models like GPT (where G stands for "Generative") are also generative models. They use more advanced architectures (transformers) rather than simple neural networks or CNNs, but the underlying concept of generative modeling is the same.

5 GAN Architecture

The GAN architecture is composed of two main components: a **Generator** and a **Discriminator**, which operate in an adversarial game.

5.1 Components

5.1.1 Training Samples (Real Images)

A high-dimensional sample space containing real images (or text, speech signals). These serve as the reference distribution for training.

5.1.2 Generator

- Takes **low-dimensional input vectors** (latent vectors) as input — these can be random noise.
- The generator has **no relation** with the training samples — it never sees the real images.
- Produces **fake images** from the latent vectors.
- Functions similarly to a **decoder** in an autoencoder: takes a low-dimensional compressed representation and generates an output image.

5.1.3 Discriminator

- Receives **both** real images (from training samples) and fake images (from the generator).
- Acts as a **binary classifier**: outputs whether an image is real or fake.
- Learns the distribution of real images to distinguish them from fake images.

5.2 The Adversarial Training Process

1. The generator creates fake images from random latent vectors.
2. Both real and fake images are fed to the discriminator.
3. The discriminator classifies each image as real or fake.
4. A **loss function** is computed based on the discriminator's performance.
5. **Alternating updates**:
   - **Update discriminator**: Lock the generator weights. Update discriminator weights using the loss to make it better at detecting fakes.
   - **Update generator**: Lock the discriminator weights. Update generator weights to produce more realistic images that fool the discriminator.
6. Repeat iteratively until the generator produces realistic-looking outputs.

Adversarial Game

The alternating training process between generator and discriminator is called an **adversarial game**. It follows the principles of **game theory** and specifically the concept of **Nash equilibrium** — the training continues until neither the generator nor the discriminator can improve further given the other's strategy.

6 GAN Loss Function

6.1 Binary Cross-Entropy Foundation

The GAN loss function is based on **binary cross-entropy** since the discriminator performs two-class classification (real vs fake).

The binary cross-entropy loss is:

$$L_D = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

where:

- y_i = actual label (1 for real image, 0 for fake image)
- p_i = probability output from the discriminator

Binary Nature of the Loss

When $y_i = 1$ (real image), the second term becomes zero, leaving only $\log(p_i)$. When $y_i = 0$ (fake image), the first term becomes zero, leaving only $\log(1 - p_i)$. The loss function operates on only one term at a time.

6.2 Discriminator Loss

The discriminator receives both real images from training data and fake images from the generator:

$$L_D = -\left[\frac{1}{N_{\text{real}}}\sum_i \log D(x_i) + \frac{1}{N_{\text{fake}}}\sum_i \log(1 - D(G(z_i)))\right]$$

where:

- x_i = real images from training data
- z_i = latent vector (input to generator)
- $G(z_i)$ = fake image generated by the generator
- $D(\cdot)$ = discriminator output probability

The discriminator aims to **maximize** this function (removing the negative sign), so it always correctly identifies real and fake images.

6.3 Generator Loss

The generator has no access to real images. Its loss function contains only the fake image term:

$$L_G = \frac{1}{N_{\text{fake}}}\sum_i \log(1 - D(G(z_i)))$$

The generator aims to **minimize** this function, meaning it wants the discriminator to fail at identifying its fake images as fake.

6.4 Combined Min-Max Objective

The overall GAN objective function combines both:

$$\min_G \max_D \; V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

- **Maximize** with respect to D: The discriminator should always identify real from fake.
- **Minimize** with respect to G: The generator should create images that the discriminator cannot distinguish from real.

7 Advantages of GANs

1. **Simple algorithm**: The GAN framework is straightforward to implement.
2. **Maximum likelihood estimation**: The training process based on binary cross-entropy produces a maximum likelihood probability estimation.
3. **Reduced overfitting**: Since the generator never sees the training data, it is less likely to overfit. If the generator had access to training samples, it would become biased toward them.

8 Challenges of GANs

8.1 Vanishing Gradient Problem

If the discriminator is initially able to perfectly identify fake images (loss function is zero), the gradient becomes zero. This means:

- Neither generator nor discriminator weights get updated.
- The generator never improves to create more realistic images.
- The system gets stuck with zero gradients.

Solution: Use a modified loss function with regularization added to the cost function (e.g., changed rule process or regularizers).

8.2 Mode Collapse

Mode collapse occurs when the generator starts producing only a **limited variety** of images rather than diverse outputs:

- Example: Given a face image, it may only generate slight variations (with glasses, without moustache) but not truly diverse versions (childhood appearance, old age appearance).
- Example: When asked to generate different animal species, it may only produce cat images if trained primarily on cats.

8.3 Solutions to GAN Challenges

1. **Modified cost function**: Use a different variant of the GAN loss function that avoids vanishing gradients and mode collapse.
2. **Mini-batch discrimination**: Rather than feeding all training samples at once, provide data in mini-batches (e.g., first cats, then dogs, then other animals) to encourage diversity.
3. **Update generator first**: Rather than updating the discriminator first, try updating the generator weights first to improve mode collapse behavior.

9 Types of GANs

9.1 Vanilla GAN

Vanilla GAN

Vanilla GAN is the most fundamental type of GAN where both the generator and discriminator are **fully connected neural networks** (FC neural networks).

- Real images are **flattened** into $M \times N$ pixel vectors and fed to the neural network discriminator.
- Latent vectors are similarly flattened and fed to the fully connected neural network generator.
- Regardless of input type (image, text, or speech), if the model uses fully connected neural networks for both generator and discriminator, it is classified as a Vanilla GAN.

9.2 DCGAN (Deep Convolutional GAN)

DCGAN

DCGAN (Deep Convolutional GAN) replaces the fully connected neural networks with **convolutional neural networks (CNNs)** in the GAN architecture.

- **Generator**: Uses **transposed convolution** (deconvolution) layers to generate images from latent vectors.
- **Discriminator**: Uses several **CNN layers** followed by **fully connected layers** to classify images as real or fake.
- The discriminator considers the distribution of real images and determines whether a given image is real or fake.

Aspect	Vanilla GAN	DCGAN
Generator	Fully connected NN	Transposed convolution
Discriminator	Fully connected NN	CNN + fully connected layers
Input handling	Flattened vectors	Spatial feature maps
Architecture	Simple MLP	Deep convolutional layers

9.3 Wasserstein GAN (WGAN)

The Wasserstein GAN modifies the standard GAN loss function. Instead of using binary cross-entropy with probability outputs, the WGAN:

- Converts the probability function into a **scoring function**.
- This scoring function is "bucketed" — a process related to the **Lipschitz function** (Lipschitz continuity constraint).
- Rather than bucketing the probability directly, the probability is transformed into a score, and that score is bucketed.
- This approach helps address the vanishing gradient and mode collapse issues present in standard GANs.

10 Summary

- **Autoencoders** compress input data through an encoder into a latent space and reconstruct it via a decoder, but produce blurry outputs due to information loss.
- **Variational Autoencoders (VAE)** extend autoencoders by producing mean (μ) and standard deviation (σ) vectors, combined with random sampling ($\epsilon \sim \mathcal{N}(0,1)$) to generate diverse outputs via $z = \mu + \sigma \cdot \epsilon$.
- **Discriminative models** classify or segment input data but cannot generate entirely new data from scratch.
- **Generative models** learn the underlying probability distribution of training data to generate new, realistic samples.
- **GANs** use an adversarial game between a generator (creates fake data) and a discriminator (classifies real vs fake), following a min-max objective based on binary cross-entropy.
- The generator never sees training data, which reduces overfitting risk.
- Key challenges include **vanishing gradients** and **mode collapse**, addressed through modified loss functions, mini-batch discrimination, and generator-first updates.
- GAN variants include **Vanilla GAN** (fully connected NNs), **DCGAN** (convolutional NNs with transposed convolution), and **Wasserstein GAN** (scoring-based loss with Lipschitz constraint).